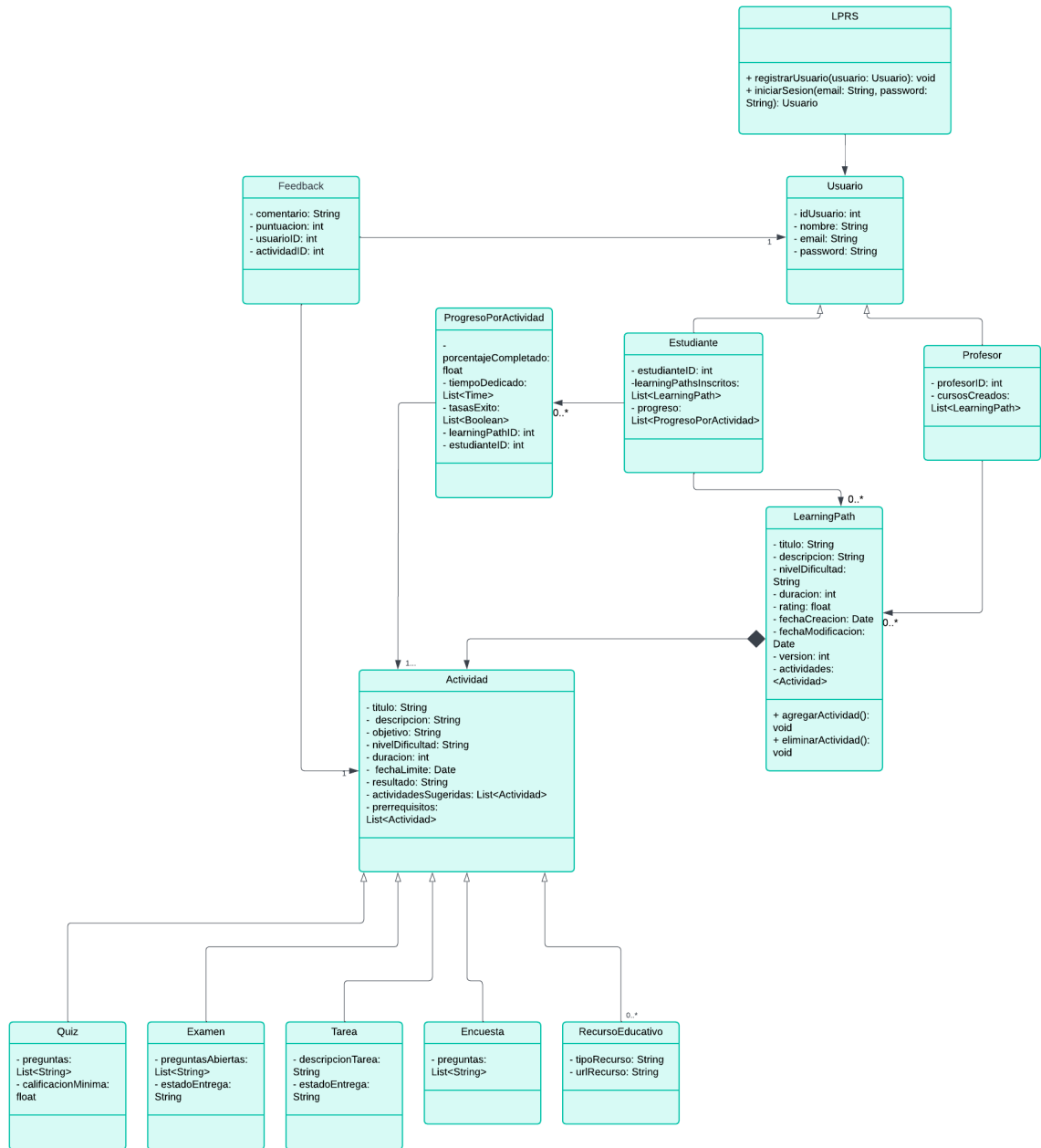
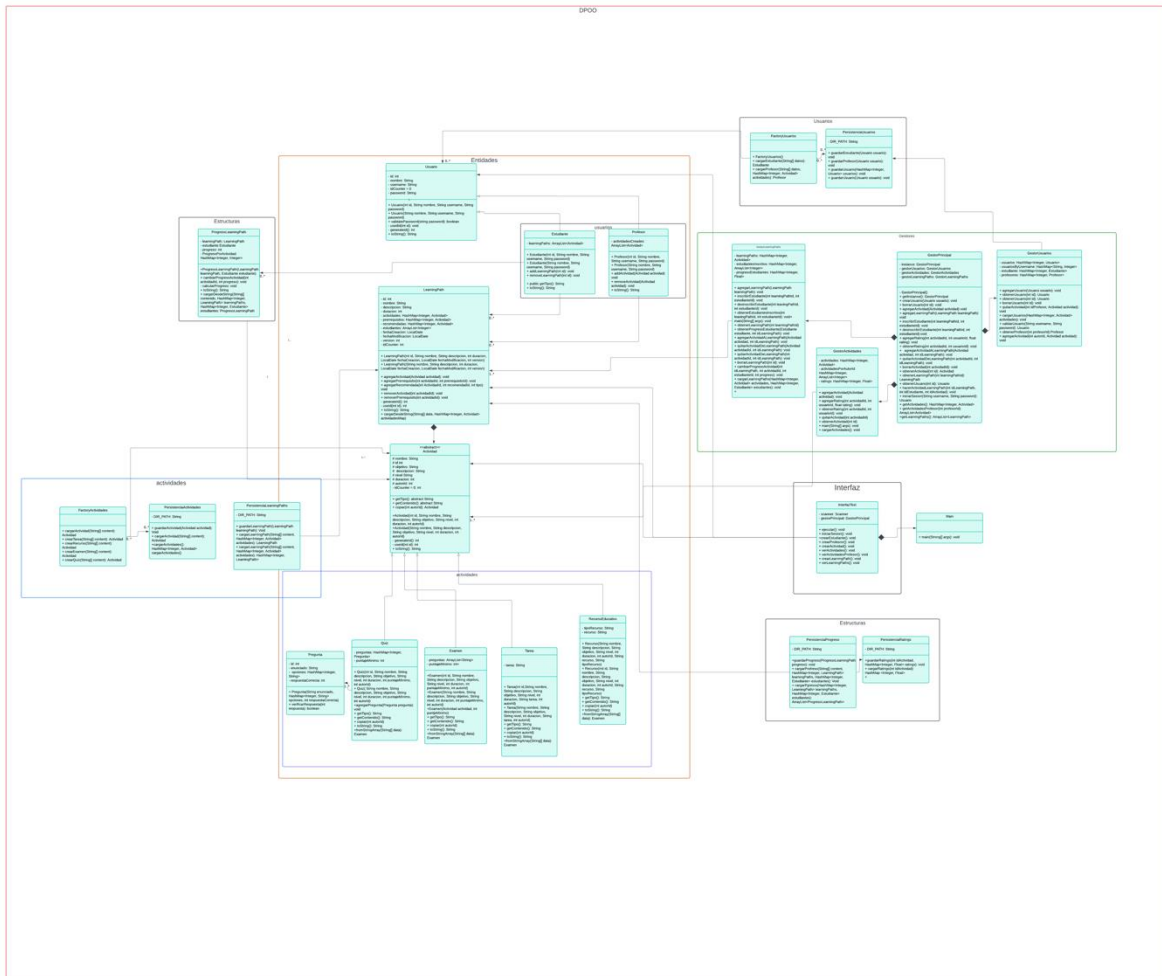






## Distribución de clases:

Se maneja un gestor principal que sirve de puente entre la interfaz y la lógica. Este gestor central, como se muestra en el diagrama UML, está conectado con los gestores de cada tipo de entidad, como el gestor de usuarios, que administra a los estudiantes y profesores; el gestor de actividades, que gestiona las actividades y sus valoraciones; y el gestor de *learningPaths*, que gestiona tanto los *learningPaths* como los usuarios inscritos en ellos. Al ser un puente, el gestor principal no contiene lógica compleja o notable; prácticamente, su función se limita a llamar a los métodos de los otros gestores.

A partir de este punto, no se detallarán en profundidad las clases usadas para manejar entidades (como usuarios o actividades), ya que el 90% de su función es almacenar información, mientras que el restante se encarga de manejar su persistencia, formateando su información para guardarla en un archivo de texto plano y leerla nuevamente. Este proceso se explicará más adelante en la sección de persistencia.

El gestor de usuarios maneja dos tipos de usuarios distintos: Estudiante y Profesor. La principal diferencia entre ellos es que el estudiante almacena referencias a los *learningPaths* en los que está inscrito, mientras que el profesor referencia las actividades que ha creado. Ambos heredan de la clase abstracta Usuario, que contiene la información básica que ambos necesitan (como *id*, nombre de usuario y contraseña). Además, el gestor de usuarios cuenta con un mapa para encontrar usuarios según su nombre de usuario (siendo la clave el nombre de usuario y el valor el *id*), y mapas adicionales para almacenar estudiantes y profesores por separado, además de un mapa general que incluye a todos los usuarios.

El gestor de actividades, como su nombre indica, se encarga de gestionar las actividades, almacenándolas en un mapa con estructura <id, actividad>, además de otro mapa para clasificar las actividades según el profesor que las creó. También maneja un mapa de mapas, donde la clave principal es la actividad y el valor es un mapa cuyo contenido asocia el *id* del usuario con el *rating* que ha asignado a la actividad, permitiendo así almacenar valoraciones por usuario y evitar duplicados.

El gestor de *learningPaths* almacena *learningPaths* (que ya contienen referencias a sus estudiantes y actividades, con prerequisites y actividades recomendadas antes y después) en un mapa similar a los anteriores, donde la clave es un *id* y el valor es el *learningPath*. Adicionalmente, tiene un mapa para clasificar a los estudiantes inscritos en cada *learningPath* y un mapa de mapas, donde la clave principal es el *id* del *learningPath* y el valor es otro mapa, con la clave como *id* del estudiante inscrito y el valor como un objeto de la clase *ProgresoLearningPath*, que almacena la información del progreso de un estudiante en cada actividad y en general del *learningPath*.

Mediante la conexión de estos gestores, se almacena la información y se crean las interacciones necesarias, ya sea inscribir a un usuario en un *learningPath* o crear una actividad y agregarla a un *learningPath*, además de calificarlas y copiarlas cuando un profesor distinto al creador original desea modificarla.

## **Persistencia:**

Para la persistencia, se utilizan archivos de texto plano con un formato similar para todos los tipos de clases. Este formato consiste en usar un archivo de texto por cada objeto individual (incluso si hay copias de actividades, cada una tiene un *id* distinto y, por lo tanto, un archivo diferente). Esto permite un guardado de información más rápido que usar un solo CSV para todas las instancias de una clase, ya que un CSV requeriría reescribir todo el archivo solo para borrar o modificar la información de un objeto guardado (en el enfoque más sencillo, dado que facilita evitar errores al borrar filas incorrectas).

El formato utilizado es de una línea por cada atributo en general, excepto en algunos atributos especiales que son mapas o listas, ya que estos pueden requerir varias líneas. Por ejemplo, para los usuarios en general se maneja una línea para el *id*, otra para el nombre y otra para el nombre de usuario; sin embargo, en el caso de un estudiante, al final del archivo se usa una línea por cada *learningPath* al que se ha inscrito. Por otro lado, para un profesor se utilizan varias líneas para referenciar las actividades que ha creado.

|            |          |
|------------|----------|
| 1          | 2        |
| Juan       | Profe    |
| juan       | profe    |
| 1234       | profe    |
| Estudiante | Profesor |
| 1          | 1        |
|            | 2        |
|            | 3        |
|            | 4        |
|            | 5        |
|            | 6        |

Para todas las entidades estándar, se utiliza este método de persistencia, mientras que para algunas estructuras más complejas se aplica un formato diferente, donde puede haber dos o tres números en una línea separados por un espacio. Este formato se usa para almacenar información de un mapa o mapa de mapas (explicados en la sección anterior), lo cual permite una carga "modular", primero cargando por separado estas entidades sencillas y luego las más complejas que requieren que las sencillas ya estén cargadas.

Para implementar esto, se crearon métodos *toString* que formatean la información para agregarla a los archivos de texto y un método que permite cargarlos a partir de estos archivos (a los cuales generalmente se les pasa como parámetro un *String[]* luego de aplicar *Split* por saltos de línea, con algún recurso adicional si es necesario). Así, solo fue necesario implementar una clase que se encarga de leer y guardar los archivos en la carpeta adecuada (todos los archivos están en la carpeta "data", junto a "src"), escribiendo en esta y leyendo de esta al momento de cargar. Estas clases usan *FileWriter* para guardar y *File.readAllBytes* para leer todo el contenido de un archivo, en lugar de recorrerlo línea por línea, facilitando así el uso de *Split*.